

Séance 2 — Organiser et répéter les traitements

Introduction à la programmation Python — IUT 1re année

Séance 2

Organiser et répéter les traitements

Fonctions, objets, séquences, textes et boucles

Un programme devient rapidement difficile à maintenir lorsqu'il répète du code ou manipule de nombreuses valeurs séparément.

À la fin de la séance, vous devez être capables de découper un programme en fonctions, d'utiliser des objets existants, de regrouper des données et de répéter automatiquement un traitement.

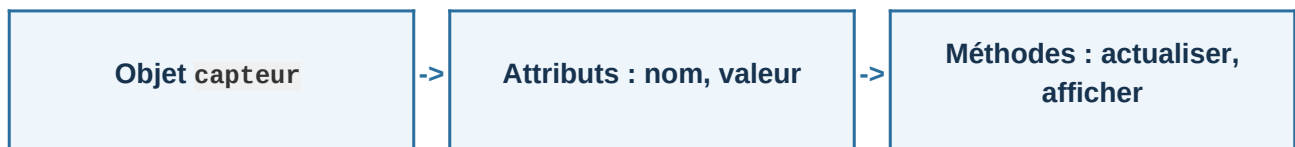
Cette séance fait passer du petit programme linéaire à un programme structuré. Les fonctions organisent les traitements, les séquences regroupent les données et les boucles permettent d'appliquer une même opération à plusieurs valeurs.

La POO rassemble données et comportements

La **programmation orientée objet** (POO) organise un programme autour d'objets qui regroupent :

- des **attributs** : les données propres à l'objet ;
- des **méthodes** : les opérations que l'objet sait effectuer.

Un objet représente ainsi une entité du programme avec un état et des comportements.



La POO est particulièrement utile lorsqu'un programme manipule plusieurs éléments de même nature : capteurs, instruments, fichiers ou graphiques.

La POO associe les données et les traitements qui leur correspondent au lieu de les disperser dans le programme. Une classe définit cette organisation commune ; les objets créés à partir d'elle portent leurs propres valeurs.

Une classe sert de modèle aux objets

Une **classe** décrit la structure commune de ses futurs objets. Pour les exemples suivants, on suppose qu'une bibliothèque fournit cette classe :

```
CapteurTemperature(nom, valeur_initiale)
```

Élément défini par la classe

Exemple

constructeur	CapteurTemperature(...)
attributs	.nom, .valeur
méthodes	.actualiser(), .afficher()

La classe constitue le modèle commun ; les valeurs particulières appartiendront à chaque objet créé.

La documentation d'une classe présente généralement son constructeur, ses attributs et ses méthodes. Elle indique les arguments attendus, les types de valeurs et les effets produits.

Instancier crée un nouvel objet

Créer un objet à partir d'une classe s'appelle une **instanciation** :

```
capteur_salle = CapteurTemperature("Salle A", 20.5)
```



`capteur_salle` est une **instance** de `CapteurTemperature`. Dans ce cours, il faudra surtout savoir instancier et utiliser des classes existantes, pas écrire leur définition.

Le constructeur s'appelle comme une fonction. Ses arguments fournissent les valeurs nécessaires à l'initialisation. L'objet obtenu possède alors les attributs et les méthodes prévus par la classe.

Chaque instance conserve son propre état

Deux appels du constructeur créent deux objets indépendants :

```
capteur_salle = CapteurTemperature("Salle A", 20.5) capteur_dehors =
CapteurTemperature("Extérieur", 12.8) capteur_salle.valeur = 21.0
print(capteur_salle.valeur) # 21.0 print(capteur_dehors.valeur) # 12.8
```

Modifier `capteur_salle` ne modifie pas `capteur_dehors`.

On accède aux attributs et aux méthodes d'un objet avec le **point** :

```
objet.attribut objet.methode()
```

Chaque instance possède son identité et son propre ensemble d'attributs. Les objets peuvent donc évoluer séparément, même lorsqu'ils ont été créés à partir de la même classe.

Les attributs décrivent l'état d'un objet

Les attributs sont des variables associées à une instance. Ils sont souvent initialisés par le constructeur :

```
capteur = CapteurTemperature("Salle A", 20.5) print(capteur.nom) # Salle A
print(capteur.valeur) # 20.5
```

Un attribut modifiable peut ensuite recevoir une nouvelle valeur :

```
capteur.valeur = 21.3 print(capteur.valeur) # 21.3
```

La documentation précise le nom, le rôle, le type et l'unité éventuelle de chaque attribut. Il ne faut pas inventer un attribut qui n'est pas prévu par la classe.

Tous les attributs ne sont pas nécessairement destinés à être modifiés directement. Certaines classes imposent de passer par une méthode afin de contrôler la nouvelle valeur ou de maintenir la cohérence de l'objet.

Les méthodes font agir l'objet

Une méthode est une fonction attachée à un objet. Elle peut consulter ou modifier ses attributs :

```
capteur = CapteurTemperature("Salle A", 20.5) capteur.actualiser(21.3)
print(capteur.valeur) # 21.3
```

Une méthode peut aussi produire un affichage ou renvoyer un résultat :

```
capteur.afficher() # Salle A : 21.3 °C
```

Dans `capteur.actualiser(21.3)`, l'objet placé avant le point est celui sur lequel la méthode agit.

La méthode reçoit implicitement l'objet concerné. Son effet et sa valeur de retour sont décrits dans la documentation. Certaines méthodes modifient l'objet, tandis que d'autres renvoient une nouvelle valeur sans changer l'état initial.

En Python, toutes les valeurs sont des objets

Les objets ne concernent pas seulement les classes fournies par une bibliothèque. Les nombres, textes et listes sont eux aussi des objets :

```
temperature = 20.5 message = "mesure terminée" mesures = [18.2, 19.1]
print(type(temperature)) # <class 'float'> print(type(message)) # <class 'str'>
print(type(mesures)) # <class 'list'>
```

Leur type détermine les opérations et méthodes disponibles :

```
message.upper() # "MESURE TERMINÉE" mesures.append(20.0) # modifie la liste
```

Nous utiliserons cette même notation avec NumPy, Matplotlib et pyserial.

Inspecter un objet

Python permet d'interroger un objet lorsque l'on ne connaît pas exactement son contenu :

```
type(capteur) isinstance(capteur, CapteurTemperature) dir(capteur)
help(capteur.actualiser)
```

Outil	Information obtenue
<code>type(objet)</code>	classe de l'objet
<code>isinstance(objet, Classe)</code>	vérification du type attendu
<code>dir(objet)</code>	noms des attributs et méthodes
<code>help(...)</code>	documentation disponible

`dir()` peut produire une longue liste contenant des noms techniques. Il sert surtout à repérer un nom, puis `help()` ou la documentation explique comment l'utiliser.

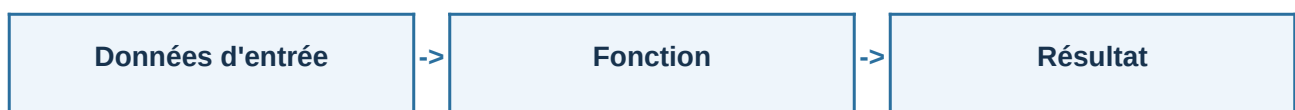
Dans un environnement comme Thonny, l'autocomplétion après le point complète cette inspection. Ces outils aident à explorer, mais la documentation officielle reste la référence pour connaître les paramètres, les effets et les valeurs renvoyées.

Organiser le code avec des fonctions

Copier-coller un même calcul à plusieurs endroits augmente le risque d'erreur et rend les modifications difficiles.

Une fonction regroupe un traitement sous un nom :

```
def convertir_c_en_k(temperature_c): return temperature_c + 273.15
temperature_k = convertir_c_en_k(20.0)
```



La définition avec `def` enregistre le traitement mais ne l'exécute pas encore. L'appel de la fonction déclenche son exécution. Un nom de fonction commence généralement par un verbe qui décrit l'action réalisée.

Paramètres et arguments

Un **paramètre** est un nom utilisé dans la définition. Un **argument** est la valeur transmise lors de l'appel.

```
def convertir_c_en_k(temperature_c): # paramètre return temperature_c + 273.15
resultat = convertir_c_en_k(20.0) # argument
```

Le paramètre permet d'utiliser le même traitement avec des valeurs différentes.

Valeurs par défaut

Un paramètre peut recevoir une valeur utilisée lorsque l'appel ne fournit pas l'argument correspondant :

```
def afficher_mesure(valeur, unite="°C"): print(f"Mesure : {valeur} {unite}")
afficher_mesure(20.5) afficher_mesure(293.65, "K")
```

Les paramètres obligatoires sont placés avant ceux qui possèdent une valeur par défaut.

Une valeur par défaut convient à un cas fréquent, mais l'appelant peut la remplacer. Les paramètres rendent une fonction générale sans l'obliger à lire directement des variables globales ou à demander elle-même les données à l'utilisateur.

Retourner ou afficher

`return` fournit un résultat au reste du programme ; `print()` produit seulement un affichage.

```
def calculer_energie(puissance, duree): return puissance * duree
energie = calculer_energie(12.0, 3.0) print(f"Énergie : {energie:.1f} Wh")
```

Le résultat retourné peut être mémorisé, comparé ou utilisé dans un autre calcul.

Une fonction sans `return` explicite renvoie la valeur spéciale `None`. `return` interrompt également l'exécution de la fonction. Séparer le calcul de l'affichage rend généralement la fonction plus facile à réutiliser et à tester.

Portée des variables

Une variable créée dans une fonction est **locale** : elle n'est accessible que pendant l'exécution de cette fonction.

```
def convertir_c_en_k(temperature_c): resultat = temperature_c + 273.15
return resultat
temperature = 20.0
```

`resultat` est locale à la fonction ; `temperature` est définie dans le programme principal.

Une fonction peut lire certaines variables définies à l'extérieur, mais cette dépendance est souvent difficile à repérer. Il est préférable de transmettre les données par les paramètres et de récupérer les résultats avec `return`. L'emploi de variables globales doit rester limité et justifié.

Regrouper des valeurs avec une liste

Une série de mesures ne doit pas être stockée dans une succession de variables indépendantes.

```
mesures = [18.2, 19.1, 20.0, 19.7]
```

Une liste est une séquence **ordonnée** et **modifiable** :

```
mesures[0] = 18.4 mesures.append(20.2)
```

Elle peut contenir un nombre variable d'éléments.

Les éléments d'une liste sont placés entre crochets et séparés par des virgules. Python autorise le mélange de types, mais une série scientifique reste généralement homogène. `append()` ajoute un élément à la fin de la liste.

Tuples et dépaquetage

Un tuple est une séquence ordonnée qui ne peut pas être modifiée après sa création.

```
position = (48.85, 2.35) latitude, longitude = position
```

Liste	Tuple
<code>[18.2, 19.1]</code>	<code>(48.85, 2.35)</code>
modifiable	non modifiable
série évolutive	ensemble de valeurs associé

L'impossibilité de modifier un tuple est appelée immutabilité. Le dépaquetage affecte chaque élément à une variable ; le nombre de variables doit correspondre au nombre d'éléments. Les tuples sont courants pour représenter des coordonnées ou retourner plusieurs résultats.

Indexer une séquence

Chaque élément possède une position appelée **index**. Le premier index vaut zéro.

```
valeur 10 20 30 40 index 0 1 2 3 négatif -4 -3 -2 -1
```

```
valeurs = [10, 20, 30, 40] premiere = valeurs[0] derniere = valeurs[-1]
```

Un index inexistant provoque une `IndexError`.

Extraire une partie de séquence

Le **slicing** sélectionne une portion sans modifier la séquence d'origine :

```
valeurs = [10, 20, 30, 40, 50] valeurs[1:4] # [20, 30, 40] valeurs[:2] # [10, 20]  
valeurs[::2] # [10, 30, 50]
```

Dans `[debut:fin]`, la borne de début est incluse et celle de fin est exclue.

Le troisième nombre éventuel indique le pas. Une borne omise signifie le début ou la fin de la séquence. Les mêmes principes s'appliquent aux listes, aux tuples, aux chaînes de caractères et, plus tard, aux tableaux NumPy.

Fonctions utiles sur les séquences

Python fournit des opérations courantes qui évitent de réécrire une boucle :

```
mesures = [18.2, 19.1, 20.0] len(mesures) # nombre d'éléments sum(mesures) # somme
min(mesures) # minimum max(mesures) # maximum sorted(mesures) # nouvelle liste
triée
```

Le test `19.1 in mesures` indique si une valeur appartient à la séquence.

`sorted()` produit une nouvelle liste et ne modifie pas la séquence fournie. À l'inverse, la méthode `list.sort()` modifie directement une liste. `sum()`, `min()` et `max()` supposent que les éléments concernés sont compatibles entre eux.

Chaînes de caractères

Une chaîne `str` est une séquence de caractères. Elle peut être indexée et découpée :

```
capteur = "TEMP-01" capteur[0] # "T" capteur[:4] # "TEMP"
```

Quelques caractères possèdent une écriture spéciale : `\n` représente un retour à la ligne et `\t` une tabulation.

Pour un chemin Windows contenant des antislashes, une chaîne brute évite de les doubler :

```
chemin = r"C:\mesures\essai1.csv"
```

Les chaînes sont non modifiables : une transformation produit une nouvelle chaîne.

Nettoyer et découper du texte

Les méthodes de chaînes préparent notamment les données lues dans un fichier :

```
ligne = " temperature;20,5 " ligne = ligne.strip() ligne = ligne.replace(",", ".")
morceaux = ligne.split(";") nom = morceaux[0] valeur = float(morceaux[1])
```

`join()` effectue l'opération inverse en réunissant plusieurs chaînes avec un séparateur.

`strip()` retire les espaces et fins de ligne aux extrémités. `replace()` remplace toutes les occurrences recherchées. `split()` découpe et renvoie une liste. La méthode `find()` recherche une sous-chaîne et renvoie son index, ou `-1` si elle est absente.

Répéter un traitement avec `for`

Une boucle `for` parcourt successivement les éléments d'une séquence :

```
mesures = [18.2, 19.1, 20.0] for mesure in mesures: print(f"{mesure:.1f} °C")
```

À chaque tour, `mesure` reçoit l'élément suivant. Le bloc indenté est exécuté une fois par élément.

Il n'est pas nécessaire de gérer manuellement l'index lorsque seule la valeur est utile. Le nom de la variable de boucle doit décrire l'élément parcouru. Une chaîne, un tuple et de nombreuses autres structures peuvent également être parcourus.

Compter et accumuler

`range()` produit une suite d'entiers, notamment pour répéter une opération un nombre connu de fois :

```
for numero in range(5): print(numero) # de 0 à 4
```

Une variable peut accumuler progressivement un résultat :

```
total = 0 for mesure in mesures: total += mesure
```

`range(debut, fin, pas)` exclut la borne de fin, comme le slicing.

Répéter avec `while`

Une boucle `while` répète un bloc tant que sa condition reste vraie :

```
tentatives = 0 while tentatives < 3: print(f"Tentative {tentatives + 1}")
tentatives += 1
```

Elle convient lorsque le nombre de répétitions dépend d'une condition. La condition doit pouvoir devenir fausse pour éviter une boucle infinie.

Modifier le parcours d'une boucle

Deux instructions permettent de modifier ponctuellement le déroulement :

- `break` arrête immédiatement la boucle ;
- `continue` passe directement au tour suivant.

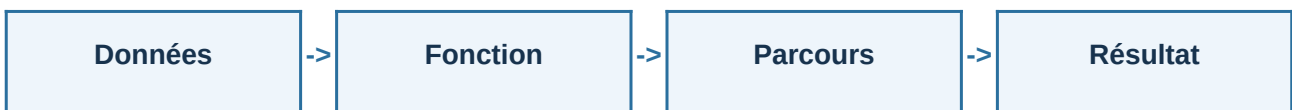
```
for mesure in mesures: if mesure < 0: continue if mesure > 30: break print(mesure)
```

Elles restent utiles lorsque leur rôle est simple et clairement identifiable.

Synthèse de la séance

Nous disposons maintenant de trois outils complémentaires :

```
organiser un traitement → fonction regrouper des données → liste, tuple, str  
répéter une opération → boucle for ou while
```



Les paramètres font entrer les données dans une fonction et `return` en fait sortir le résultat. Les séquences regroupent plusieurs valeurs et les boucles les parcourent. Ces outils permettront, à la séance suivante, de lire et d'analyser des données conservées dans des fichiers.

Exercices — Séance 2 : Organiser et répéter les traitements

Ex. 1 — Fonction sans `max()` Écrire une fonction `maximum(liste)` qui renvoie le plus grand élément d'une liste non vide, sans utiliser `max()`, à l'aide d'une boucle.

Ex. 2 — Fonctions de conversion Écrire une fonction `convertir_depuis_m(valeur, unite="m")`. La valeur d'entrée est exprimée en mètres et l'unité de sortie peut être `"m"`, `"cm"` ou `"km"`. La fonction doit retourner le résultat et signaler une unité inconnue.

Ex. 3 — Tuple et dépaquetage Créer un tuple de coordonnées GPS (`latitude, longitude`), le dépaqueter dans deux variables, et afficher un message formaté.

Ex. 4 — Découpage de texte À partir d'une phrase saisie par l'utilisateur, afficher : les 5 premiers caractères, les 5 derniers, un mot sur deux (via `split()` et slicing de liste).

Ex. 5 — Nettoyage de texte À partir d'une chaîne du type `" Alice ; Bob ;Charlie "`, obtenir la liste `["Alice", "Bob", "Charlie"]` sans espaces superflus, en utilisant `strip()`, `split()` et une boucle.

Ex. 6 — Recherche avec `while/break` Écrire un programme qui parcourt une liste de mesures avec `while` et s'arrête (`break`) dès qu'une valeur dépasse un seuil, en affichant sa position (utiliser `continue` pour ignorer les valeurs négatives). Veiller à faire avancer l'indice à chaque tour, y compris avant `continue`.